工程專案與研究歷程說明　資料快照：2026-10-04

AI 自適應學習平台 技術型高中版

系統現況與開發歷程

這個專案從我父親——一位技術型高中數學老師——每天遇到的教學問題開始。最早只是競賽中的 AI 數學教學系統原型；得獎後，我持續改版與重構，逐步把教材、數學題目、學生練習、AI、判題與學習紀錄整合進同一套系統，並把它帶回課堂使用。

目前已進入真實課堂使用。115 學年第一學期起，系統於 2 個技術型高中班級正式上線，共 50 名學生；截至 2026-10-04，累積 1,224 次實際作答，平均答對率 67%。這些紀錄來自學生實際練習，不是測試資料。系統仍持續開發，教材匯入、GenCode、判題器與自適應複習功能仍持續改進。團隊另有 RL 模型自適應研究，由另一位成員主責，與我主導的正式系統工程開發分開進行。

2 個班級　**50 名學生**　**1,224 次實際作答**　**67% 平均答對率**

多三甲 31 人｜資一乙 19 人｜真實學生使用紀錄｜資料快照 2026-10-04

開發歷程

競賽原型	扶輪盃 AI 第一名，完成第一個能運作的版本。
重構擴充	育秀盃金獎後，持續補上教材、題目、介面與後端。
進入課堂	開始讓學生實際使用，問題也從這裡開始出現。
正式上線	115 學年第一學期起，2 班、50 名學生實際使用。
持續開發	教材匯入、GenCode、判題器、自適應複習功能持續改進；RL 模型研究由另一位成員主責。



▲ **網站教師首頁（正式上線）** 兩個真實班級、50 名學生，以及教材、技能與 AI 的管理入口。

▶ 公開唯讀 Demo ↗ mathb.math-adaptive.org/review-demo　唯讀展示，不會寫入正式資料

我一開始想解決什麼問題？

我的父親是技術型高中數學老師，有二十多年教學經驗。一開始，我還不確定老師真正需要什麼，所以直接問他：「如果真的拿到班上，你會不會用？」「哪裡不好用？」「真正需要的是什麼？」系統是在開發、試用與進班過程中，依實際遇到的問題逐步完成。

1　一整冊教材，難道每一道題都要人工輸入？

後來做成教材匯入系統，整理成冊、章、節、技能與例題資料。

2　如果題目都是固定的，學生怎麼持續練習？

後來把「一題」變成「一支會出題的程式」。

3　AI 可以產生題目，但我怎麼知道它沒有錯？

後來建立生成、檢查、修正、測試到發布的流程。

4　學生做數學需要公式、手寫、作圖與提示，普通表單不夠。

後來做成數學練習介面，並加入 AI 助教。

分工｜葉陽甫

系統架構與程式開發、前後端、教材匯入流程、出題程式（GenCode）、數學判題、AI 整合、資料庫、測試與除錯、正式部署與維護。

父親／技高數學教師

提供教學需求與課堂情境、課本資料匯入與整理，試用系統並回饋哪裡不好用。

團隊研究支線｜RL 模型自適應研究（主要由另一位團隊成員負責）

在系統開發過程中，團隊另一位成員進一步研究：能否讓強化學習模型根據學生作答狀態決定下一步安排，並和固定規則方法比較。這條研究線嘗試以 AKT、PPO 與 RAG 探索學習狀態推估、學習路徑決策與補救內容檢索。我在此專案中主要負責正式系統架構、教材匯入、程式化出題、判題、AI 整合、部署與維護；RL 模型研究主要由團隊另一位成員負責。

後面介紹的「自適應複習模式」是正式系統功能；RL 模型研究則由團隊另一位成員主責，目前尚未整合進正式系統。

教材匯入：從 PDF／Word 到結構化教材資料

數學教材同時包含冊、章、節、技能、例題、隨堂練習、習題、自我評量，以及公式、特殊符號與圖形。如果匯入後這些關係消失，之後就沒辦法可靠地產題、判題，也找不到學生在哪個技能出錯。若直接將 PDF 交給 AI 處理，這些結構與來源資訊容易遺失。

因此我設計了一套教材匯入與整理流程：同時讀取 Word 與 PDF，把教材拆成冊、章、節、技能與每一道題，並保留題目來源。



▲ **匯入系統 V3**　選課綱、年級、冊別，上傳 Word 與 PDF。Word 保留結構與公式，PDF 保留完整視覺。



▲ **例題管理中心**　匯入後拆成章、節、技能與每一道題，每題保留原始來源（如「1-1 習題 基礎題 1」）與出題程式狀態。

匯入時要保住的四件事

01｜保留章節結構

02｜辨識數學公式

03｜保留教材圖片

04｜記住每題來源

技術型高中數學 B1・B2・B3・B4

四冊教材進入同一套資料架構，每一道題都知道自己來自哪一冊、哪一章、哪一節與哪個技能。目前仍有部分題目尚未完成練習化，各章仍分別驗收。

模型建立出題程式，驗證後才發布

傳統題庫保存「題目＋答案」；我的做法是先讓 AI 模型依教材例題與題型規格，建立一支 Python 出題程式。程式經驗證後，可反覆改變數值與條件，持續產生同類型的變化題，並同步計算答案。我把這個做法叫做「程式即內容（Code-as-Content）」。

流程｜模型建立程式，程式反覆出題

1. **教材例題與題型規格**
   整理題目的數學結構與可變條件
2. **AI 建立 Python 出題程式**
   依規格產生 Generator 草稿
3. **驗證後發布**
4. **程式反覆產生變化題**
   每次改變參數，產生新的同類題
5. **同步算出答案，交給判題器**



▲ **同一支程式，不同的題目**　教材例題 4404（已上線）

但是，模型產生的程式不能直接發布。

驗證流程｜每一步都對應一種實際遇到的錯誤

1. **生成草稿**
2. **實際執行（隔離環境）**
3. **隨機產題試跑**
4. **題目與答案檢查**
5. **自動修正常見錯誤（AST Healer）**
6. **pytest 回歸測試**
7. **人工驗收**
8. **發布到學生端**

這是「絕對值不等式」單一技能於 2026-08-12 的驗收結果；其他教材仍分別驗收。



▲ **驗證與發布結果（正式系統）**　10 個元件，10 通過、0 失敗、10 發布。

模型先建立出題程式；程式通過驗證後，後續變化題由程式反覆產生，不需每題重新呼叫 AI。

數學計算與語言教學的分層設計

學生做數學需要公式、手寫、作圖與提示。我把系統分成兩層：數學計算由程式負責，語言解釋與引導由 AI 助教負責。

第一層｜數學計算

Domain・Generator・判題器

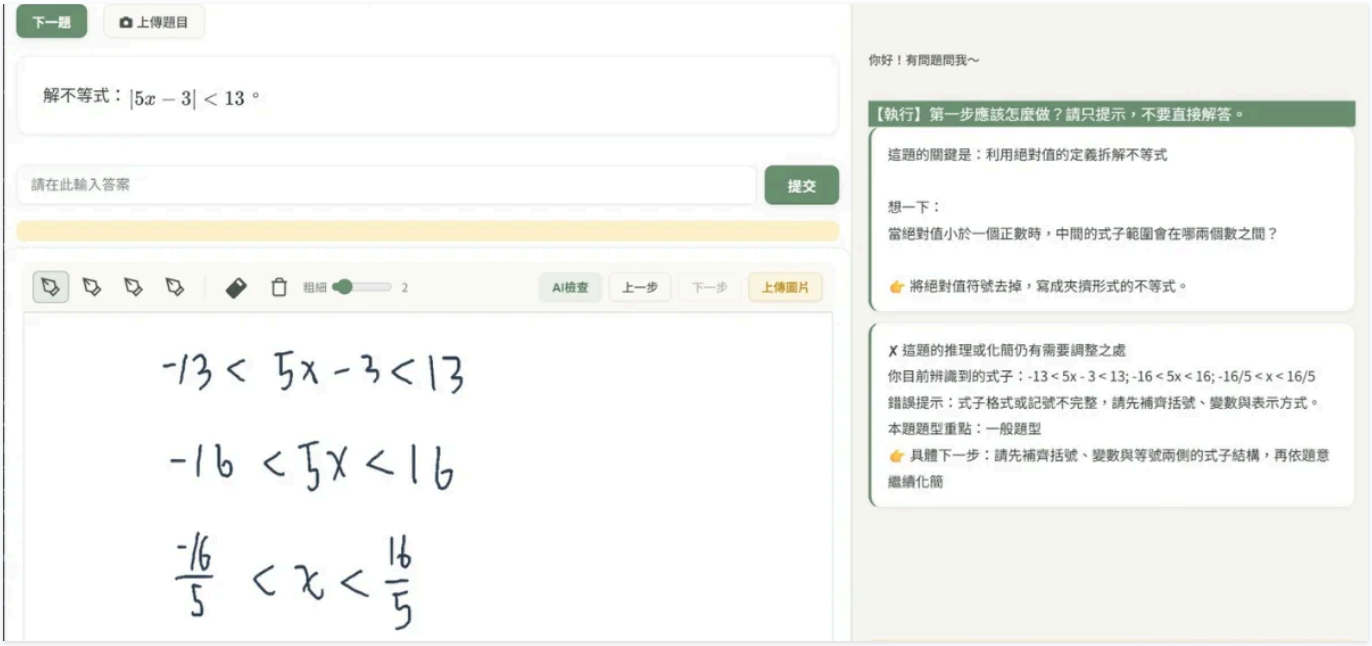
我們讓模型先把教材題型轉成可執行的 Python 出題程式。每次程式改變數值或條件產生新題時，也會同步依相同參數計算出該題的正確答案，再交由判題器比對學生作答。題目與答案來自同一套可執行規則，不由 AI 分別生成，因此數學邏輯可以透過測試反覆驗證。

Python 出題程式 → 產生變化題＋同步計算答案 → 判題器

第二層｜語言教學

AI 助教

解釋題意、分段提示、分析學生錯在哪，用觀察、聯想、執行三步提問引導學生自己想。

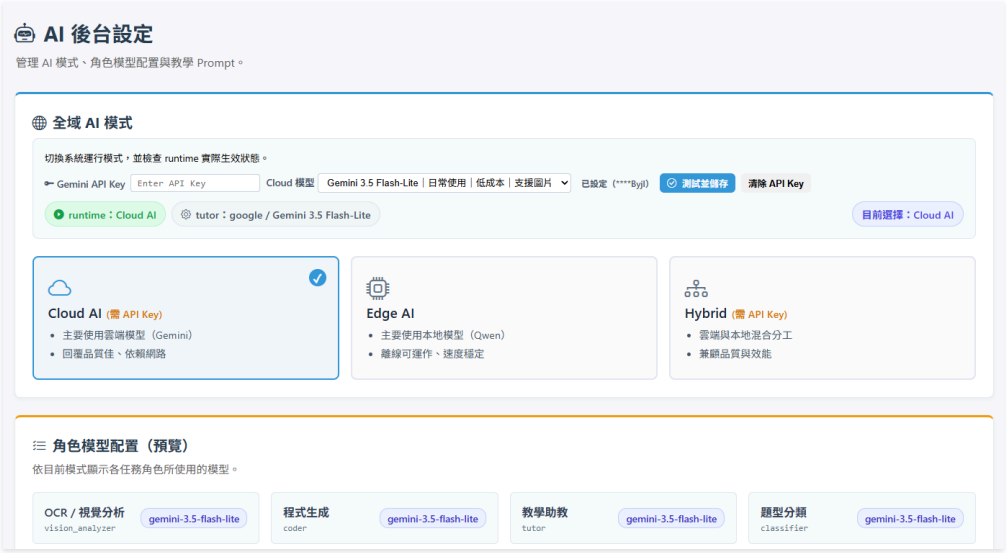


▲ **學生練習介面（正式網站）** 題目、答案欄與手寫畫布由程式出題與判題；右側 AI 助教讀了學生的手寫算式，只給提示，回覆前程式會先檢查有沒有洩漏答案。

正式練習時，題目、標準答案與判分主要由第一層負責，AI 不直接當作數學答案的來源。我將數學計算與語言生成分開處理，以降低生成式 AI 影響數學正確性的風險。AI 助教的提示仍可能出錯，因此提示內容需要持續檢查。

模型調度與部署考量

學校和 Demo 不一樣：少數人試用時，每次都呼叫大型雲端模型看不出問題；數十位學生長期使用後，API 費用、回應速度與資料外傳都會變成實際問題。所以我讓系統可以依任務選擇模型，不綁定單一模型。



▲ **AI 後台設定（正式系統）** Cloud／Edge／Hybrid 全域切換，依角色配置模型。

能力

教材理解、程式生成與多模態分析需要較強的雲端模型；簡單提示與固定範圍的互動，小型本地模型有機會取代部分雲端呼叫。

成本與延遲

雲端模型有 API 成本與網路延遲；本地模型沒有 API 費用，但取決於主機效能與推論速度。

隱私

可在本機完成的互動留在本機，減少作答與學習資料不必要的外傳。這只能減少外傳，無法絕對保證。

三種模式

Cloud 全用雲端；Edge 全用本機；Hybrid 簡單任務先在本機完成，需要時再交給雲端。

已實作

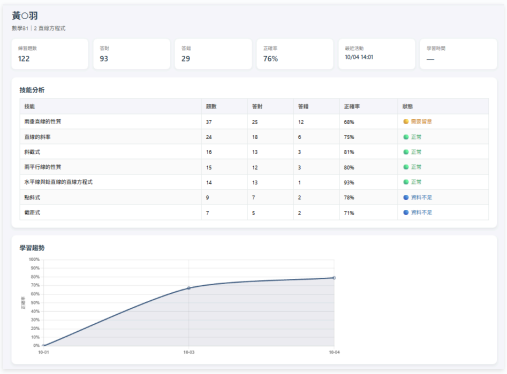
雲端模型串接（Gemini）、Cloud／Edge／Hybrid 全域切換、依角色配置模型、本地模型（Qwen）選項、本地主機部署。

仍在驗證

不同本地模型的品質與速度、混合調度與 fallback 策略、成本與品質的取捨。目前尚未證明小型模型足以取代雲端。

模型選擇要依任務需求考量能力、延遲、成本與資料傳輸。程式生成可以接受較高延遲，但學生在等待提示時，回應速度就很重要；涉及學生作答與學習紀錄時，也要考慮這些資料是否真的有必要送到雲端。Cloud、Edge 與 Hybrid 的配置會依能力、延遲、成本與資料傳輸需求調整。

從學習紀錄到自適應複習



▲ **學生個人學習分析（正式網站，2026-10-04）** B1「直線方程式」122 題、76%。姓名已遮蔽。

每一次作答都留下題型、技能、正確率與時間紀錄，累積成學生的學習狀態。

技能拆開看

把每個數學技能分開分析，標出需要留意、正常與資料不足。

看見變化

比較不同時間的正確率，看學生是否隨練習進步。

這些紀錄供學生與教師查看，在部分單元中也用於自適應複習的決策。

正式系統功能｜規則式自適應複習模式（與團隊另一位成員主責的 AKT／PPO／RAG 研究支線分開）

1 → **主線題目**　2 → **學生作答**　3 → **更新能力值與錯誤紀錄**　4 → **判斷卡點**　5 → **前置技能補救**　6 **返回主線**

未形成卡點 → 繼續主線



▲ **自適應教學正式系統**　同一頁整合學習導航、作答狀態、題目、補救提示與 AI 助教。

補救觸發：穩定卡點｜補救依據：前置技能｜完成條件：返回原本主線。目前已在部分單元實作此流程；跨單元擴充、評量與教學銜接及學習成效驗證仍持續進行。

真實課堂使用與正式部署

真實使用後出現的三個問題

1 題目本身缺參數

學生遇到要求 a＋b，但題目沒定義 b。

→ 修正題目規格，加入回歸測試。

2 答案正確卻被判錯

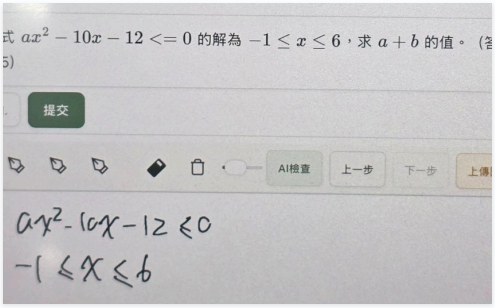
分數、小數、區間、LaTeX 等價答案，不能只比字串。

→ 答案正規化＋數學等價判定。

3 作圖題不是填充題

答案在數線、坐標或手寫區，不能硬轉成文字。

→ 建立獨立作答類型與判題器。



▲ 現場錯題：要求 a＋b，卻只有參數 a。

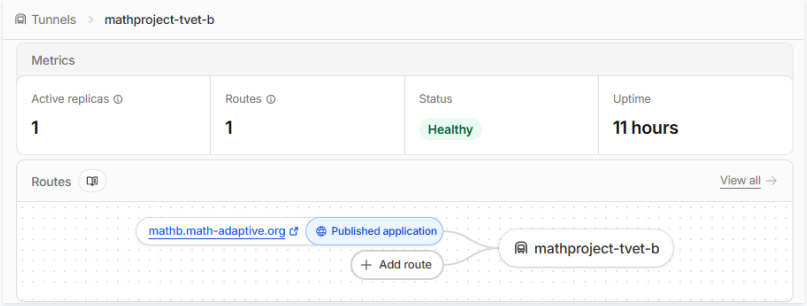


▲ 實際進班使用。

2 個班級、50 名學生、1,224 次作答，都是學生使用留下的真實紀錄。課堂中實際遇到的錯誤，後續都加入回歸測試，避免同類問題再次發生。

從 localhost 到外部正式服務

一開始系統只跑在我自己的電腦上。要讓學生從學校或家裡連進來，我才開始處理主機、網路、網域與外部連線。現在正式主機放在家中書房，透過 HiNet 與 Cloudflare Tunnel 對外提供服務。



◀ **Cloudflare Tunnel（真實後台）**
Tunnel「mathproject-tvet-b」Healthy，正式網域 mathb.math-adaptive.org 已接上；主機 IP、Replica ID 與主機名稱已裁除。

學生／老師 → Cloudflare（DNS＋Tunnel）→ HiNet 500M → 家中書房主機 → Flask → 教材・題庫・資料庫・AI

目前限制　正式主機只有這一台，同時使用人數只驗證到 2 個班級。主機：AMD Ryzen 7 5700F、32 GB RAM、RTX 5060 Ti 16 GB。

▶ GitHub｜系統程式碼與版本紀錄 ↗　github.com/yyfsunnyboy/Mathproject_tvet_mathB

現在做到哪裡，下一步要證明什麼？

1｜已上線的一般系統

教材匯入、程式化出題、判題、AI 助教、教師端、學習紀錄與正式網站；2 個班級、50 名學生、1,224 次實際作答。

2｜已實作｜自適應複習模式

部分單元已能依作答紀錄、系統內部學習能力值與前置技能關係，在主線、補救與返回主線之間動態切換；各單元完整度仍不一致。

3｜團隊研究支線｜RL 模型自適應研究（主要由另一位團隊成員負責）

團隊另一位成員主要探索 AKT 學習狀態推估、PPO 學習路徑決策與 RAG 補救內容；目前尚未正式整合進前述自適應複習模式。

4｜尚未完成

團隊 RL 模型研究與正式系統的整合、跨教材泛化、評量與教學銜接、學習成效驗證，以及國中、普高與技高數學 A・C 的擴充。

下一階段核心問題：系統是否能改善學生的學習成效？

1 可重現的實驗
不同模型、策略與版本可重複比較，並保留實驗紀錄。

2 擴大系統測試
從約 50 人，測試多人同時使用的穩定性、效能與資料一致性。

3 前後測與分組設計
建立前測、後測與可比較的分組，避免把練習後進步直接當成系統的效果。

4 資料分析
比較補救前後進步幅度、完成技能時間、同類錯誤是否重複。

5 研究倫理與資料治理
學習歷程與作答內容的匿名化、同意與資料使用規範。

6 擴大使用場域
進入更多班級與學習情境，檢驗結果能否重現。

團隊模型研究 → 與正式系統整合 → 擴大場域 → 實驗驗證 → 教育成效

目前尚未完成學習成效驗證，列為下一階段研究項目。

AI 自適應學習平台（技術型高中版）　▶ 公開唯讀 Demo ↗　mathb.math-adaptive.org/review-demo